

Cost Breakdown

GA Prompt Optimiser for LLM Code Generation

Complete API Cost Analysis — Google Cloud Vertex AI

Group 1

Computational Intelligence — Spring 2026

April 30, 2026

All prices sourced from the official Anthropic/Google pricing pages, April 2026.
Backend: Google Cloud Vertex AI (Gemini 2.5 Flash & Pro).
Charges are real GCP billing — the \$300 free credit applies.

Contents

Chapter 1

Pricing Sources

All prices are sourced from Anthropic’s official API pricing page (April 2026) [?] and verified against Google Cloud Vertex AI pricing [?]. Vertex AI rates for Gemini models match the direct API rates.

1.1 Model Pricing (Vertex AI — Standard)

Model	Input (\$/MTok)	Output (\$/MTok)	Role
Gemini 2.5 Flash (<code>gemini-2.5-flash</code>)	0.15	0.60	Target (code
Gemini 2.5 Pro (<code>gemini-2.5-pro</code>)	1.25	10.00	Optimizer (op
Gemini 2.0 Flash (<code>gemini-2.0-flash</code>)	0.10	0.40	(reference onl
Claude Haiku 4.5 (<code>claude-haiku-4-5-20251001</code>)	1.00	5.00	(previous bac
Claude Sonnet 4.6 (<code>claude-sonnet-4-6</code>)	3.00	15.00	(previous bac

Table 1.1: Vertex AI pricing (USD per million tokens). MTok = million tokens. Claude models listed for comparison with the previous CLI-based design.

Why two Gemini models?

Gemini 2.5 Flash generates C++ code (cheap, deterministic at temperature 0) — this is the model being *optimised*. **Gemini 2.5 Pro** performs crossover and mutation (creative, temperature 0.7) — this is the evolutionary *operator*. Both models are called via the `google-genai` Python SDK with service account authentication — no interactive login or API key is required. Despite Pro costing 8× more per output token, it accounts for under 30% of the total bill because it is used rarely (66 calls vs. 4,425 Flash calls).

Thinking tokens are billed at the output rate

Gemini 2.5 models produce internal chain-of-thought (*thinking tokens*) before the visible response. These tokens are **not shown** in the response text but are reported in `usage_metadata.thoughts_token_count` and billed at the standard output token rate. The project tracks and records them in `calls.jsonl`. Budget: 512 thinking tokens for Flash (fitness), 2048 for Pro (operators). Actual usage is typically 200–400 thinking tokens for Flash and 800–1500 for Pro. The output token figures in this document **include thinking tokens**.

1.2 Cost Comparison: Previous vs. Current Backend

Version	Target	Optimizer	Pop/Gen/Pool	Estimated cost
Previous (CLI-based)	Claude Haiku 4.5	Claude Sonnet 4.6	15/20/100	~\$27.67
Current (Vertex AI)	Gemini 2.5 Flash	Gemini 2.5 Pro	5/12/65	~\$3.01

Table 1.2: Backend and parameter changes between versions. Gemini Flash is $6.7\times$ cheaper per input token and $8.3\times$ cheaper per output token than Claude Haiku. The smaller GA configuration (Pop=5, Gen=12, Pool=65) further reduces calls by $\sim 16\times$.

Chapter 2

Token Measurements

Token sizes are estimated from real Codeforces problem data. Codeforces problems are shorter than APPS problems; typical problem statement lengths are 500–2000 characters. The `thinking_token_count` contribution is estimated from observed Gemini 2.5 behaviour with the configured budgets.

Call type	Input (tokens)	Output (tokens)	Breakdown
Fitness (Flash, code gen)	400	700	system 120 + question 280 / code 400 + t
Crossover (Pro)	220	1280	meta+parents 220 / merged 80 + thinking
Mutation (Pro, any type)	220	1280	meta+parent 220 / mutated 80 + thinking

Table 2.1: Estimated token sizes per API call. Output includes thinking tokens (billed at output rate).

Input token breakdown (fitness calls)

- **System prompt** (chromosome): ~ 70 tokens (seed prompts are 3–5 sentences, ~ 280 words ≈ 70 tokens).
- **_CODE_GEN_DIRECTIVE**: ~ 50 tokens (operational constraint appended at call time, never stored in chromosome).
- **Total system**: ~ 120 tokens.
- **User message** (question + instruction): Codeforces problem text ~ 230 tokens + “Write a complete C++ program...” instruction ~ 50 tokens = ~ 280 tokens.
- **Total input per fitness call**: ~ 400 tokens.

Output token breakdown (fitness calls)

- **C++ code** (visible): Codeforces solutions average 50–150 lines at ~ 8 tokens/line ≈ 400 tokens visible output.
- **Thinking tokens** (budget=512, typical usage ~ 300): ~ 300 tokens (billed at output rate, *not* in response text).
- **Total output per fitness call**: ~ 700 tokens.

Output token breakdown (Pro operator calls)

- **Visible response** (merged/mutated prompt): ~ 80 tokens (a prompt paragraph of ~ 3 – 5 sentences).
- **Thinking tokens** (budget=2048, typical usage ~ 1200): ~ 1200 tokens.
- **Total output per Pro call**: ~ 1280 tokens.

Chapter 3

API Call Counts

3.1 GA Configuration (Current)

Parameter	Value
POPULATION_SIZE	5
GENERATIONS	12
ELITISM_COUNT	0 (no frozen elites)
CROSSOVER_ONLY_SLOTS	2 (first 2 offspring per gen: crossover only)
EVOLUTION_SIZE	65 (stratified: 5 harder + 60 regular)
HOLDOUT_SIZE	200
CROSSOVER_RATE	0.80
MUT_INJECT_PROB	0.25
MUT_DELETE_PROB	0.15
MUT_REPHRASE_PROB	0.10

Table 3.1: GA hyperparameters driving call count calculations.

3.2 Call Count Derivation

Phase	Model	Expected calls	Formula
Gen-0 fitness	Flash	325	$5 \text{ pop} \times 65 \text{ problems}$
Gen 1–12 fitness	Flash	3 900	$5 \text{ offspring} \times 12 \text{ gen} \times 65 \text{ problems}$
Holdout evaluation	Flash	200	$1 \text{ best} \times 200 \text{ problems}$
Total Flash		4 425	
Crossover	Pro	48	$60 \text{ offspring} \times 0.80$
Mutation: inject	Pro	9	$36 \text{ mut} - \text{eligible} \times 0.25$
Mutation: delete	Pro	5	36×0.15
Mutation: rephrase	Pro	4	36×0.10
Total Pro		66	
Grand Total		4 491	

Table 3.2: API call counts by phase (no early convergence stop).

Key assumptions

- Population size = 5, ELITISM_COUNT = 0 $\Rightarrow$ all 5 slots are bred fresh each generation.
- Total offspring across 12 generations = $5 \times 12 = 60$.
- All 60 offspring are eligible for crossover (rate = 0.80) $\Rightarrow 60 \times 0.80 = 48$ crossover calls.
- Only offspring beyond CROSSOVER_ONLY_SLOTS=2 are mutation-eligible: $3 \text{ slots/gen} \times 12 \text{ gen} = 36$ mutation-eligible offspring.

- Mutation probabilities are independent: inject 0.25, delete 0.15, rephrase 0.10. Expected calls: $36 \times 0.25 + 36 \times 0.15 + 36 \times 0.10 = 9 + 5.4 + 3.6 = 18$ mutation calls.
- Holdout evaluation runs in parallel using `ThreadPoolExecutor` (200 calls, all Flash).
- Early convergence can reduce generations below 12; counts above assume no early stop.

CROSSOVER_ONLY_SLOTS reduces operator calls

CROSSOVER_ONLY_SLOTS=2 means 2 of 5 offspring per generation skip all mutations. Across 12 generations this saves $24 \times (0.25 + 0.15 + 0.10) = 12$ expected Pro calls — a small but non-negligible saving given Pro’s high output token cost.

Chapter 4

Full Cost Breakdown

4.1 Token Totals

Phase	Calls	Input tokens	Output tokens	Total tokens
Flash — fitness (gen 0 + gens 1–12)	4 225	1 690 000	2 957 500	4 647 500
Flash — holdout evaluation	200	80 000	140 000	220 000
Pro — crossover	48	10 560	61 440	72 000
Pro — mutations (inject+delete+rephrase)	18	3 960	23 040	27 000
Flash subtotal	4 425	1 770 000	3 097 500	4 867 500
Pro subtotal	66	14 520	84 480	99 000
Grand total	4 491	1 784 520	3 181 980	4 966 500

Table 4.1: Token totals by phase. Output tokens **include thinking tokens** (billed at output rate).

4.2 Cost by Component

Component	In tokens (M)	Out tokens (M)	In cost	Out cost	Total
Flash (fitness + holdout)	1.770	3.098	\$0.27	\$1.86	\$2.13
Pro (operators)	0.015	0.084	\$0.02	\$0.84	\$0.86
Grand Total	1.785	3.182	\$0.28	\$2.70	\$2.99

Table 4.2: Cost breakdown by model. All prices in USD. Flash rate: \$0.15/\$0.60 per MTok. Pro rate: \$1.25/\$10.00 per MTok.

Where the money goes

71% of the cost is Flash output tokens (C++ code + thinking) at \$1.86. Although Flash is cheap per token, it is called 4,425 times. **28% of the cost is Pro output tokens** (\$0.84) despite only 66 calls — because Pro has a large thinking budget (2048) and its output rate is 16.7× higher than Flash output. Input tokens are negligible: only \$0.28 combined (<10% of total cost).

Component	Cost
Gen-0 fitness (5 seeds $\times$ 65 problems $\times$ Flash)	\$0.228
Gen 1–12 fitness (5 offspring $\times$ 65 problems each)	\$0.228 per gen
Genetic operators (crossover + mutations per gen)	\$0.052 per gen
Holdout (200 problems, runs once at end)	\$0.101
Gen-0 total	\$0.228
Per-generation cost (gens 1–12)	\$0.280
Holdout (one-time)	\$0.101
Full run total (12 gens + holdout)	\$2.99

Table 4.3: Cost breakdown by run phase. Gen-0 evaluates seeds sequentially with `seed_NN.json` checkpoints; no operator calls in gen 0.

Metric	Value
Total API calls	4 491
Total input tokens	1 784 520
Total output tokens	3 181 980
Total input cost	\$0.28
Total output cost	\$2.70
Grand total cost	\$2.99 $\approx$ \$3.01
Estimated wall-clock time	$\sim$ 3–5 hours
Cost per generation (1–12)	$\sim$ \$0.28
Cost per Flash call	$\sim$ \$0.00067
Cost per Pro call	$\sim$ \$0.013

Table 4.4: Grand summary — full 12-generation run, no early stop.

4.3 Cost per Generation

4.4 Summary Table

Comparison with previous backend

The old design used Claude Haiku (\$1.00/\$5.00 per MTok) + Claude Sonnet (\$3.00/\$15.00 per MTok) via the Claude CLI subprocess. The configuration was Pop=15, Gen=20, Pool=100. Estimated total: \$27.67.

The new design uses Gemini 2.5 Flash (\$0.15/\$0.60 per MTok) + Gemini 2.5 Pro (\$1.25/\$10.00 per MTok) via Google Cloud Vertex AI. Configuration: Pop=5, Gen=12, Pool=65. Estimated total: \$3.01.

Savings: $\sim$ \$24.66 (89% reduction) — driven jointly by the cheaper Gemini Flash rates and the smaller GA configuration.

Chapter 5

Thinking Token Impact

5.1 What Are Thinking Tokens?

Gemini 2.5 Flash and Pro produce internal chain-of-thought reasoning (*thinking*) before generating the visible response. Thinking tokens:

- Are **not** visible in the response text.
- Are reported in `usage_metadata.thoughts_token_count`.
- Are **billed at the same rate as output tokens**.
- Are tracked by `cost_tracker.py` and included in `calls.jsonl`.

The project configures thinking budgets via `ThinkingConfig(thinking_budget=N)`:

- **Flash (fitness)**: `thinking_budget=512`. Typical actual usage: 200–400 tokens.
- **Pro (operators)**: `thinking_budget=2048`. Typical actual usage: 800–1500 tokens.

5.2 Thinking Token Cost Contribution

Model	Calls	Thinking toks/call	Total thinking toks	Thinking cost
Flash (budget=512, avg ~300)	4 425	300	1 327 500	\$0.80
Pro (budget=2048, avg ~1200)	66	1 200	79 200	\$0.79
Total thinking cost				\$1.59

Table 5.1: Thinking token contribution to total cost. Thinking tokens account for ~53% of the total \$3.01 bill.

Thinking tokens dominate the bill

Without thinking tokens, the total cost would be approximately:

- Flash output without thinking: $4,425 \times 400 \times \$0.60/MTok = \$1.06$
- Pro output without thinking: $66 \times 80 \times \$10.00/MTok = \0.053
- Total without thinking: $\approx \$1.42$ (vs. \$3.01 with thinking)

The thinking budget is configurable. Setting `thinking_budget=0` disables thinking for both models and halves the estimated cost, at the expense of potentially lower code generation and operator quality.

Chapter 6

Optimisation Options

6.1 Cost Reduction Levers

1. **Reduce evolution pool size** ($65 \rightarrow 30\text{--}50$ problems): Proportionally reduces Flash fitness calls. 50 problems would save $\sim 23\%$ on Flash costs.
2. **Reduce generations or population:** With `POPULATION_SIZE=5` and `GENERATIONS=12`, there is limited room here without hurting GA quality. The convergence detection (CV threshold) already terminates early when the population converges.
3. **Disable or reduce thinking budget:** Setting `thinking_budget=0` halves the estimated cost to $\sim \$1.40$. This trades reasoning quality for cost.
4. **Use Gemini 2.0 Flash** ($\$0.10/\0.40 per MTok) for fitness calls: Would reduce Flash cost by 33%. Gemini 2.0 Flash does not support thinking tokens, so output cost per call drops to $400 \times \$0.40 / MTok = \0.00016 . Flash fitness cost: $4,425 \times \$0.00016 = \0.71 (vs. $\$2.13$ with 2.5 Flash).
5. **Early convergence stop:** The GA already implements CV-based early stopping. If the population converges at generation 8, cost is roughly 67% of the full run.

6.2 Alternative Configurations

Config	Pop	Gen	Pool	Flash calls	Cost	Savings
Current (baseline)	5	12	65	4 425	\$3.01	—
Reduced pool	5	12	40	2 920	\$2.00	34%
No thinking (Flash 2.5)	5	12	65	4 425	\$1.50	50%
Gemini 2.0 Flash target	5	12	65	4 425	\$1.06	65%
No thinking + 2.0 Flash	5	12	65	4 425	\$0.79	74%
Early stop (avg gen 8)	5	8	65	3 025	\$2.03	33%

Table 6.1: Cost estimates for alternative configurations. Pro operator costs are small enough to be rounded into the total.

Recommended low-cost option: Gemini 2.0 Flash target (\$1.06)

Replace `TARGET_MODEL = "gemini-2.5-flash"` with `TARGET_MODEL = "gemini-2.0-flash"` in `config.py`. Gemini 2.0 Flash has no thinking tokens, so output cost drops to $\$0.40/MTok$. Estimated cost: $\sim \$1.06$ for Flash + $\sim \$0.86$ for Pro operators = $\sim \$1.92$ total. This is a one-line change; all other infrastructure remains identical.

Chapter 7

Runtime Cost Tracking

The `cost_tracker.py` module logs every API call's exact token count and computes the real cost in real time. Three separate files are updated during the run:

1. `run_*/calls.jsonl` — streaming JSONL, one line per API call, written immediately after each call (no buffering).
2. `run_*/costs_live.json` — live running-total summary, overwritten after every call; shows up-to-date spend during a long run.
3. `Costs/cost_<run_name>.json` — same live summary written to the `Costs/` directory, named after the run directory (e.g. `cost_run_20240501_143022_tournament.json`). Updated after every API call so you can monitor spend during a long run without opening the run directory.

At run end, `tracker.flush()` writes the final summary:

`Costs/cost_<TIMESTAMP>.json`:

```
{
  "grand_total_estimated_usd": 2.99,
  "total_llm_calls": 4491,
  "total_input_tokens": 1784520,
  "total_output_tokens": 3181980,
  "breakdown_by_phase": {
    "fitness": { "calls": 4425, "input_tokens": 1770000,
                  "output_tokens": 3097500, "total_cost_usd": 2.125 },
    "crossover": { "calls": 48, "input_tokens": 10560,
                   "output_tokens": 61440, "total_cost_usd": 0.628 },
    "mutate_inject": { "calls": 9, "input_tokens": 1980,
                      "output_tokens": 11520, "total_cost_usd": 0.118 },
    "mutate_delete": { "calls": 5, "input_tokens": 1100,
                      "output_tokens": 6400, "total_cost_usd": 0.065 },
    "mutate_rephrase": { "calls": 4, "input_tokens": 880,
                        "output_tokens": 5120, "total_cost_usd": 0.052 }
  },
  "breakdown_by_model": {
    "gemini-2.5-flash": { "calls": 4425, "total_cost_usd": 2.125 },
    "gemini-2.5-pro": { "calls": 66, "total_cost_usd": 0.863 }
  }
}
```

Each run creates a **new file** (timestamped) — prior runs are never overwritten. This gives a complete cost history across all experiments.

Resume and cost continuity

When `main.py` is invoked with `-resume`, it calls `tracker.restore_from_jsonl(run_dir / "calls.jsonl")` to replay all existing call records and restore running totals. The final cost summary therefore reflects the entire run — both the pre-crash portion and the resumed portion — with no double-counting.

Individual JSONL entries in `calls.jsonl` look like:

```
{"t": "14:32:07", "phase": "fitness", "model": "gemini-2.5-flash",  
  "in": 412, "out": 693, "cost": 0.000477, "total_calls": 47,  
  "running_cost": 0.022419}  
{"t": "14:33:51", "phase": "crossover", "model": "gemini-2.5-pro",  
  "in": 228, "out": 1340, "cost": 0.013685, "total_calls": 48,  
  "running_cost": 0.036104}
```

The `out` field includes both visible response tokens and thinking tokens. The `cost` field is computed from the PRICING table in `cost_tracker.py` using the exact token counts reported by the SDK.

Bibliography

- [1] Anthropic, “Pricing — Claude API Docs,” <https://docs.anthropic.com/en/docs/about-claude/pricing>, accessed April 2026.
- [2] Google Cloud, “Vertex AI pricing — Generative AI,” <https://cloud.google.com/vertex-ai/generative-ai/pricing>, accessed April 2026.
- [3] X. Sécheresse, J.-Y. Guilbert-Ly, and A. Villedieu de Torcy, “GAAPO: Genetic algorithmic applied to prompt optimization,” *Frontiers in Artificial Intelligence*, vol. 8, Art. no. 1613007, Sep. 2025, <https://doi.org/10.3389/frai.2025.1613007>.
- [4] Q. Guo *et al.*, “Connecting large language models with evolutionary algorithms yields powerful prompt optimizers,” in *Proc. ICLR*, 2024.